ORMs in MYSQL and PostgreSQL

Yes, similar to MongoDB where Mongoose is used for Object Document Mapping (ODM), both MySQL and PostgreSQL have similar libraries and tools that you can use for Object Relational Mapping (ORM). These libraries provide higher-level abstractions for database interactions, making it easier to work with SQL databases without writing raw SQL queries all the time.

MySQL and PostgreSQL ORM options:

1. **Sequelize (for MySQL and PostgreSQL)**:
    - **Sequelize** is one of the most popular ORMs for both MySQL and PostgreSQL. It provides a higher-level abstraction that allows you to interact with the database using JavaScript objects rather than writing raw SQL queries.
    - It supports models, migrations, associations (relations), and other common database operations in a more structured way.
    - To use Sequelize:
        1. **Install** it: npm install sequelize mysql2 (for MySQL) or npm install sequelize pg pg-hstore (for PostgreSQL).
        2. **Define models** for your database tables.
        3. Perform CRUD operations without writing raw SQL.

Example of Sequelize model for User in PostgreSQL:

js

CopyEdit

```
const { Sequelize, DataTypes } = require('sequelize');

const sequelize = new Sequelize('postgres://user:password@localhost:5432/mydb'); // For PostgreSQL


const User = sequelize.define('User', {
  email: {
    type: DataTypes.STRING,
    allowNull: false,
    unique: true,
  },
  password: {
    type: DataTypes.STRING,
    allowNull: false,
```

```
  }
});

// Sync with the database (create tables if they don't exist)
sequelize.sync();

// Example of creating a user
async function createUser() {
  await User.create({ email: 'example@example.com', password: 'password123' });
}

createUser();
```

2. **TypeORM (for MySQL and PostgreSQL)**:
    - **TypeORM** is another powerful ORM for both MySQL and PostgreSQL. It works with TypeScript (though it also supports JavaScript) and allows for more structured and powerful object-relational mapping. TypeORM supports migrations, associations, and query builders.
    - To use TypeORM:
        1. **Install** it: npm install typeorm mysql2 (for MySQL) or npm install typeorm pg (for PostgreSQL).
        2. Define entities (models) for your database.
        3. Use TypeORM's repository and entity manager to handle data.

Example of a basic TypeORM entity:

ts

CopyEdit

```
import { Entity, PrimaryGeneratedColumn, Column } from 'typeorm';

@Entity()
export class User {
  @PrimaryGeneratedColumn()
  id: number;
```

```
  @Column()
  email: string;

  @Column()
  password: string;
}
```

3. **Objection.js (for PostgreSQL and MySQL)**:
    - **Objection.js** is a SQL-friendly ORM built on top of the **Knex.js** query builder. It allows you to perform SQL-like queries while still using models to handle interactions with your database.
    - Objection.js is more flexible than Sequelize and TypeORM but requires you to write more SQL-like code for certain operations.
    - To use Objection.js:
        1. **Install** it: npm install objection knex pg (for PostgreSQL) or npm install objection knex mysql2 (for MySQL).
        2. Define models and use the Knex.js query builder.

Example using Objection.js:

```
js
CopyEdit
const { Model } = require('objection');
const knex = require('knex');
const db = knex({
  client: 'pg',
  connection: 'postgres://user:password@localhost:5432/mydb', // For PostgreSQL
});

Model.knex(db);

class User extends Model {
  static get tableName() {
    return 'users';
  }
```

```
}

// Fetch user by email
async function getUserByEmail(email) {
  const user = await User.query().findOne({ email });
  console.log(user);
}

getUserByEmail('example@example.com');
```

4. **Prisma (for MySQL and PostgreSQL)**:
    - **Prisma** is a next-gen ORM for Node.js and TypeScript that allows you to work with MySQL, PostgreSQL, and SQLite in a type-safe manner.
    - Prisma generates a client based on your schema, and you interact with the database using this generated client. It's particularly useful if you want type-safety in TypeScript.
    - To use Prisma:
        1. **Install** it: npm install prisma @prisma/client.
        2. Define your Prisma schema.
        3. Run prisma migrate to generate migrations and apply them.
        4. Use the Prisma Client for querying the database.

Example of Prisma schema:

prisma

CopyEdit

```
datasource db {
  provider = "postgresql"
  url      = env("DATABASE_URL")
}

generator client {
  provider = "prisma-client-js"
}
```

```
model User {
  id       Int    @id @default(autoincrement())
  email    String @unique
  password String
}
```

Prisma usage example:

```
js
CopyEdit
const { PrismaClient } = require('@prisma/client');
const prisma = new PrismaClient();

async function createUser() {
  const user = await prisma.user.create({
    data: {
      email: 'example@example.com',
      password: 'password123',
    },
  });
  console.log(user);
}

createUser();
```

To summarize:

- **Mongoose** is specific to MongoDB, and it helps with working with NoSQL databases.
- For **SQL databases** like MySQL and PostgreSQL, you can use libraries like **Sequelize**, **TypeORM**, **Objection.js**, or **Prisma** for object-relational mapping.

Using these ORMs allows you to avoid writing raw SQL queries for basic operations, making it easier to interact with your database through models and higher-level abstractions.